Tutorial 10

▼ Overview

Tutorial (More More More Graphs

- Kahoot
- Heaps
- Tries

Lab (From Page)

- Exam Environment

▼ Major Announcements

- AT2 Due Friday 8pm
- Final Tut (Well done for almost completing COMP2521)
- Fill out my experience, 60% completion and you get some hints for the exam.

▼ Heaps

▼ What is a heap?

A heap is another way of storing numbers, similar to a BST there are two branches a right and a left but they will be sorted such that the largest number is at the top and all numbers below are less than.

They are also represented by an array where the first node is at index 1 and the children are at index 2n and 2n+1 where n is the parent. They must be inserted in the next index over in the array hence are balanced until the last row which has nodes filled from left to right. This is known as the heap property.

They can be ordered as a max or min heap.

▼ Consider the series of heap operations below. Assuming that higher values have higher priority, show the state of the heap (in both binary tree form and array form) after each operation.

```
Heap h;
Item it;
h = heapNew();

heapInsert(h, 10);
heapInsert(h,  5);
heapInsert(h, 15);
heapInsert(h , 3);
heapInsert(h, 16);
heapInsert(h, 13);
heapInsert(h,  6);
it = heapDelete(h);
heapInsert(h,  2);
it = heapDelete(h);
it = heapDelete(h);
it = heapDelete(h);
it = heapDelete(h);
it = heapDelete(h);
```

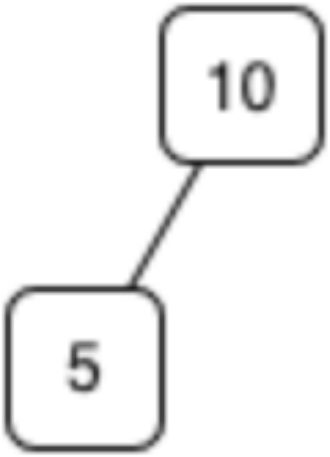
▼ Heap Activity OS Half

Operation	0	1	2	3	4	5
heapNew()	-	-	-	-	-	-

▼ heapInsert(h, 10);

Operation	0	1	2	3	4	5
	-	10	-	-	-	-

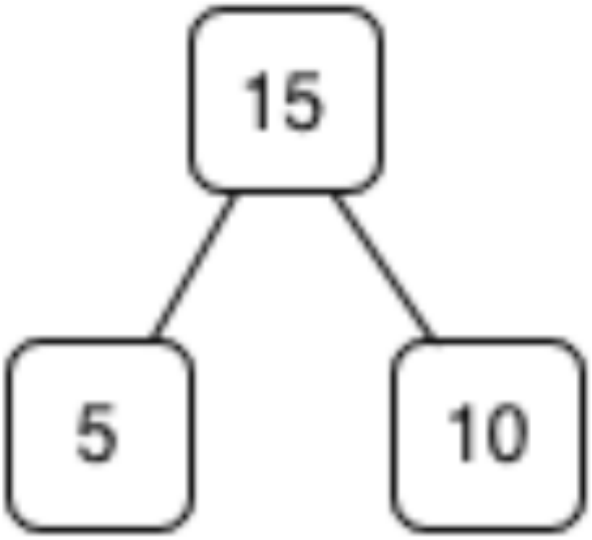
▼ heapInsert(h, 5);



Operation	0	1	2	3	4	5
	-	10	5	-	-	-

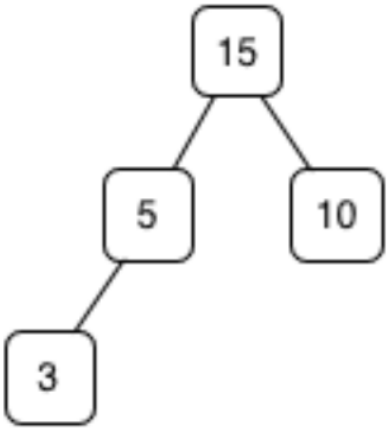
▼ heapInsert(h, 15);

Swaps the 10 and the 15 by swapping abs(n/2) where n is the current cell with n



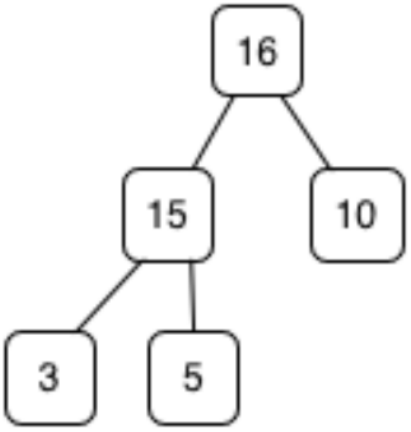
Operation	0	1	2	3	4	5
	-	15	5	10	-	-

▼ **heapInsert(h, 3);**



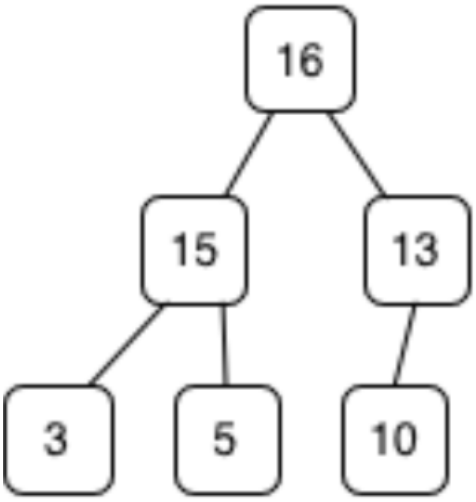
Operation	0	1	2	3	4	5
	-	15	5	10	3	-

▼ **heapInsert(h, 16);**



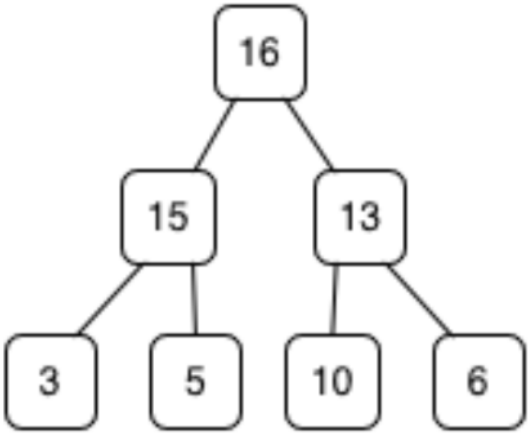
Operation	0	1	2	3	4	5
	-	16	15	10	3	5

▼ **heapInsert(h, 13);**



Operation	0	1	2	3	4	5
	-	16	15	13	3	5

▼ **heapInsert(h, 6);**



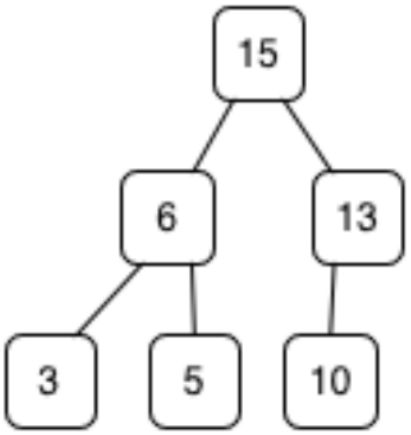
Operation	0	1	2	3	4	5
	-	16	15	13	3	5

▼ How Does Deletion Work

Take the last element, replace it with the first, then delete the last element.

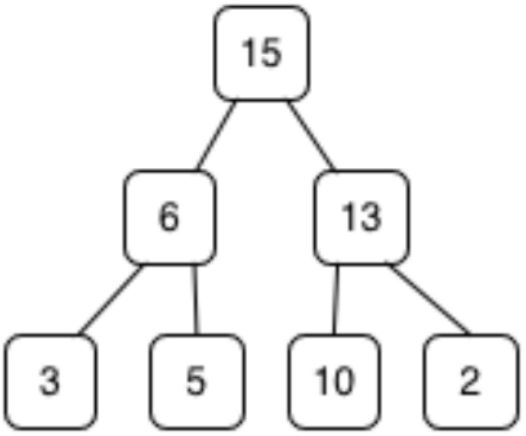
- Fix down, swap the numbers with the greatest child

▼ it = heapDelete(h);



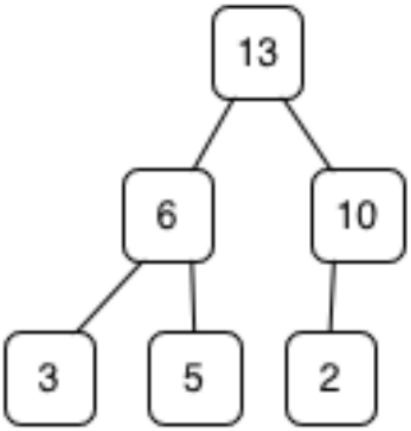
Operation	0	1	2	3	4	5
	-	15	6	13	3	5

▼ heapInsert(h, 2);



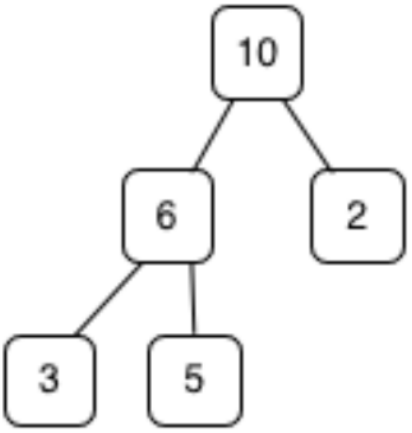
Operation	0	1	2	3	4	5
	-	15	6	13	3	5

▼ it = heapDelete(h);



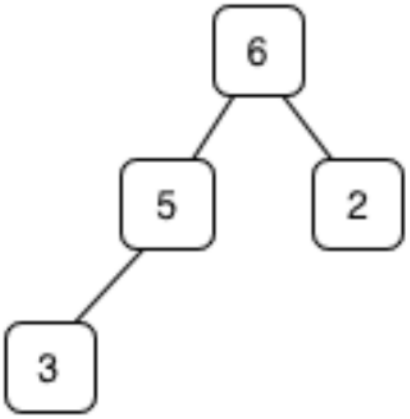
Operation	0	1	2	3	4	5
	-	13	6	10	3	5

▼ **it = heapDelete(h);**



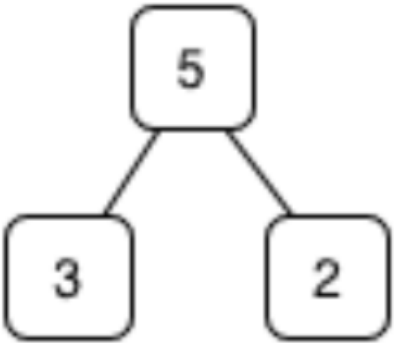
Operation	0	1	2	3	4	5
	-	10	6	2	3	5

▼ **it = heapDelete(h);**



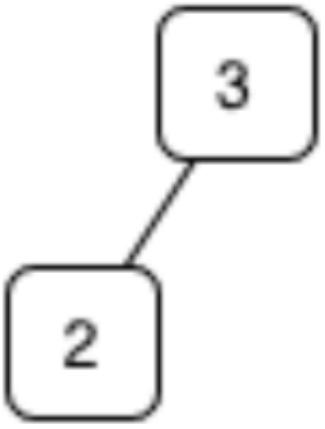
Operation	0	1	2	3	4	5
	-	6	5	2	3	-

▼ **it = heapDelete(h);**



Operation	0	1	2	3	4	5
	-	5	3	2	-	-

▼ **it = heapDelete(h);**



Operation	0	1	2	3	4	5

	-	3	2	-	-	-

▼ **Suppose you are given an array A of size n, and you would like to find the largest k elements $(k \leq n)$. For example, if given the array $A = [7, 4, 2, 5, 9]$ and the value $k = 3$, the largest k values are 9,5,7**

▼ **A simple way to solve this problem is to sort the array in increasing order, and then the largest k elements would be the last k elements of the sorted array. What is the worst-case time complexity of this approach?**

O(nlogn) it takes at least nlogn to sort the array which would dominate.

▼ **Come up with a more efficient method that uses a heap. What is the worst-case time complexity of this method?**

▼ Obvious approach

Insert into a max heap then delete the elements, but this is O(nlogn) inserts and O(klogn) deletes not faster.

▼ Faster approach

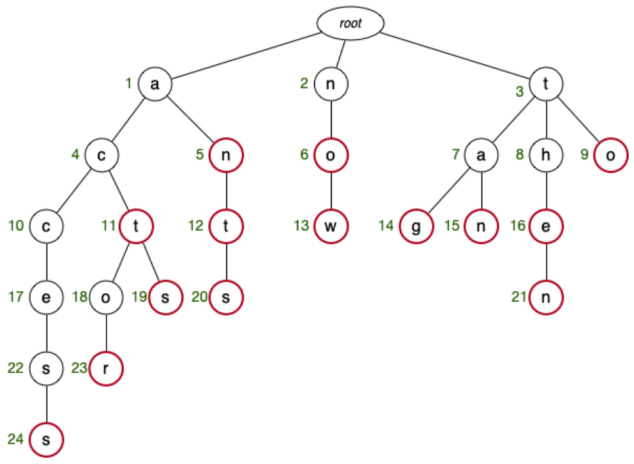
Use a min heap, insert k elements then insert one more and delete then insert then delete.

This is O(nlogk) as the heap always contains at most k+1 elements and insertion and deletion is therefore logk

▼ Tries

▼ What is a Trie

A trie is a tree like data structure that is used to store letters or subgroups of words to allow for quick retrieval of words, e.g. used in autocomplete or spellcheck.



Show which nodes would be visited when searching for each of the following words:

▼ `now`

→ 2, 6, 13

▼ `the`

→ 3, 8, 16

▼ `ant`

→ 1, 6, 13

▼ `access`

→ 1, 4, 10, 17, 22, 24

▼ `actor`

→ 1, 4, 11, 18, 23

▼ `action`

→ 1, 4, 11, fails (no "i")

▼ `actors`

→ 1, 4, 11, 18, 23, fails (leaf)

▼ `tang`

→ 3, 7, 15, fails (leaf)

▼ `a`

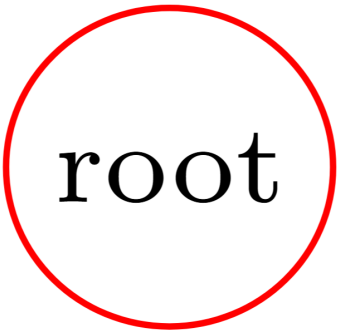
→ 1, fails (not red)

▼ Under what circumstances would inserting a new word into a trie not result in adding any new nodes? What would be the effect on the tree of inserting the word?

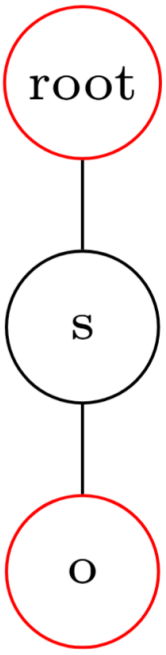
If the new word is a prefix of an existing word in the trie, then it won't require a new node to be added. The node representing the last character in the word will simply be turned into a finishing (red) node.

▼ Show the final trie resulting from inserting the following words into an initially empty trie.

`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

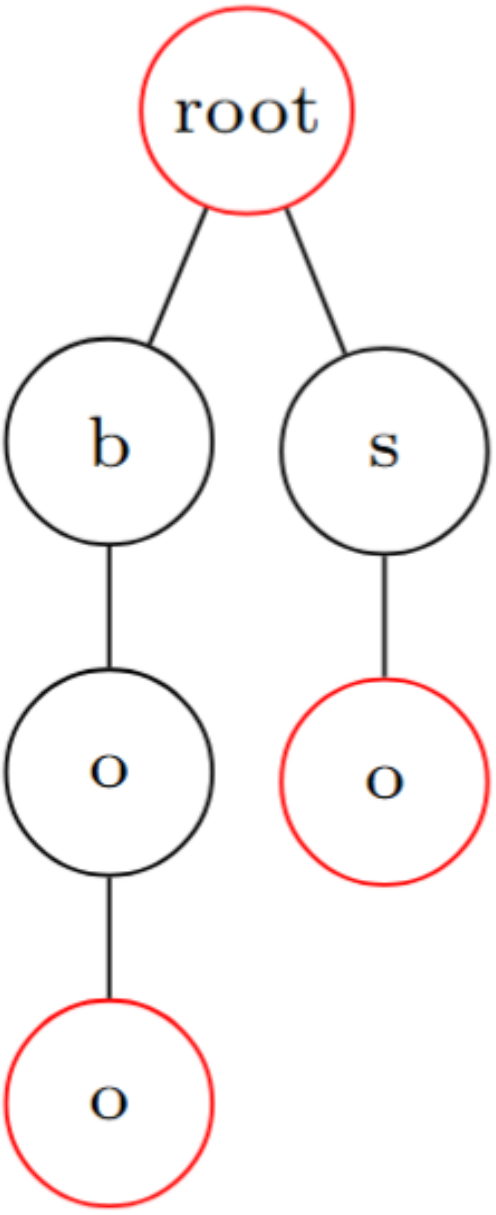


▼ Insert So



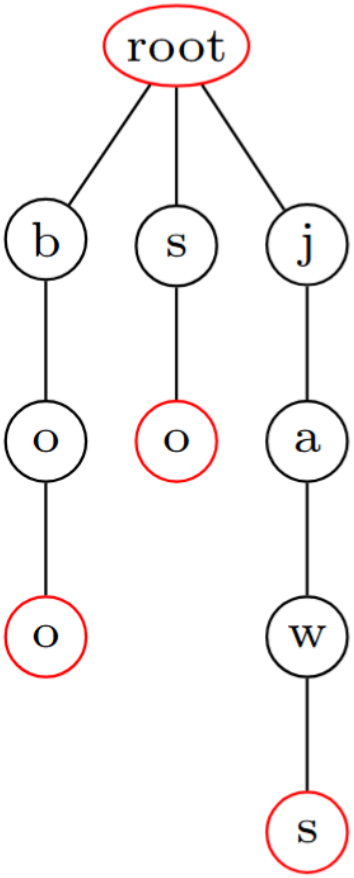
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ Insert Boo



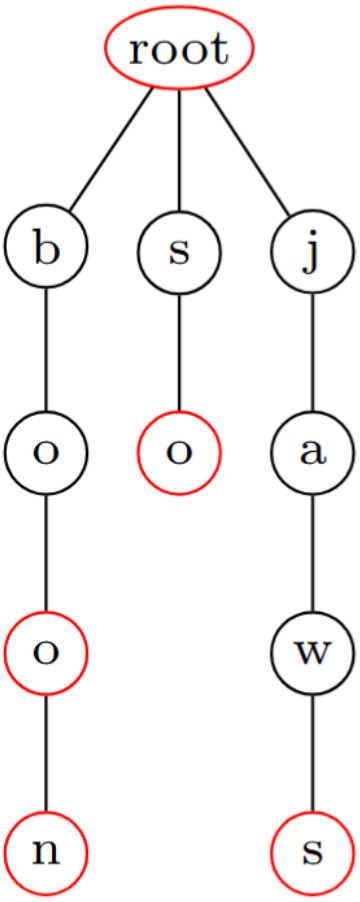
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ **Insert Jaws**



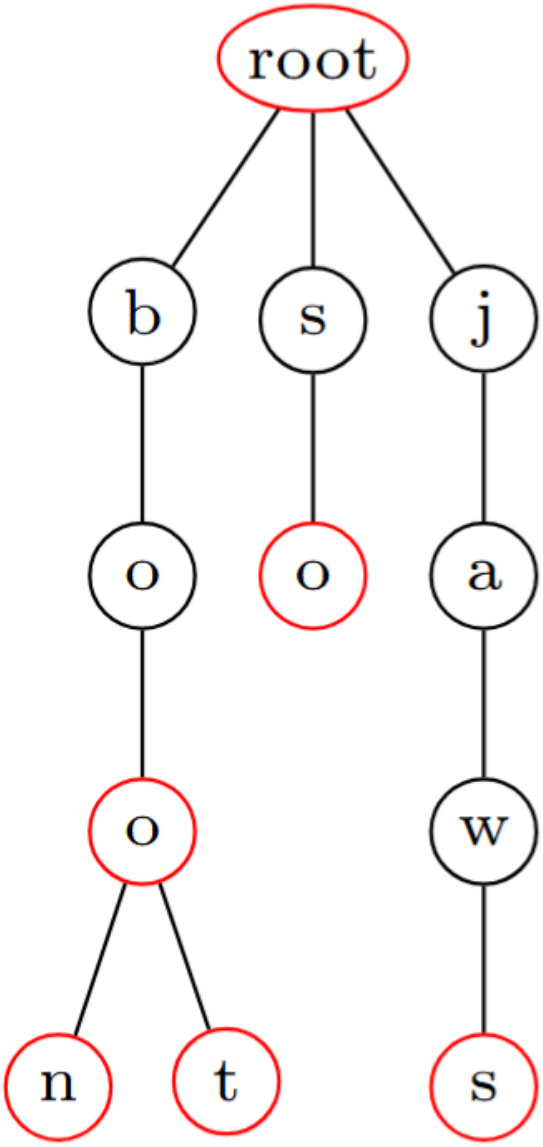
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ Insert Boon



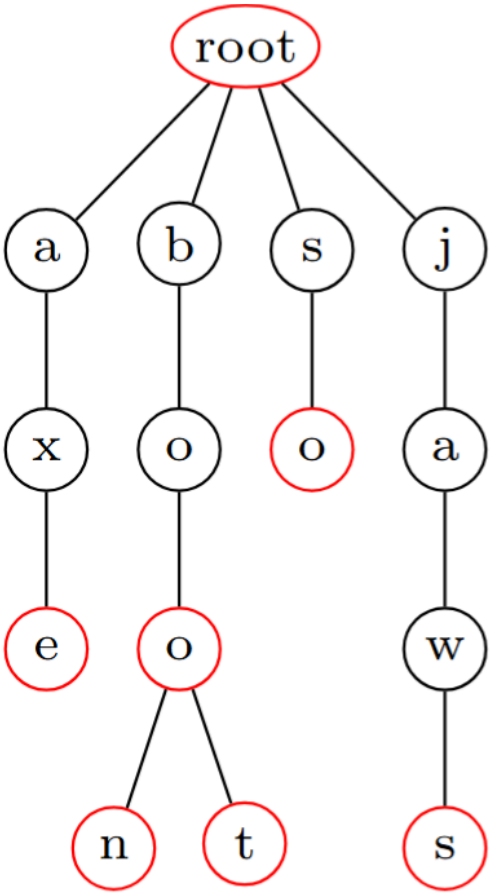
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ Insert Boot



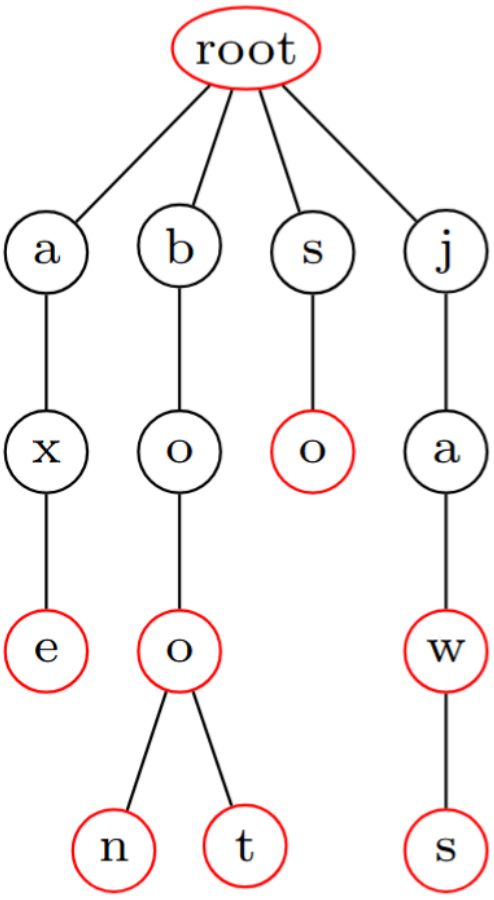
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ **Insert Axe**



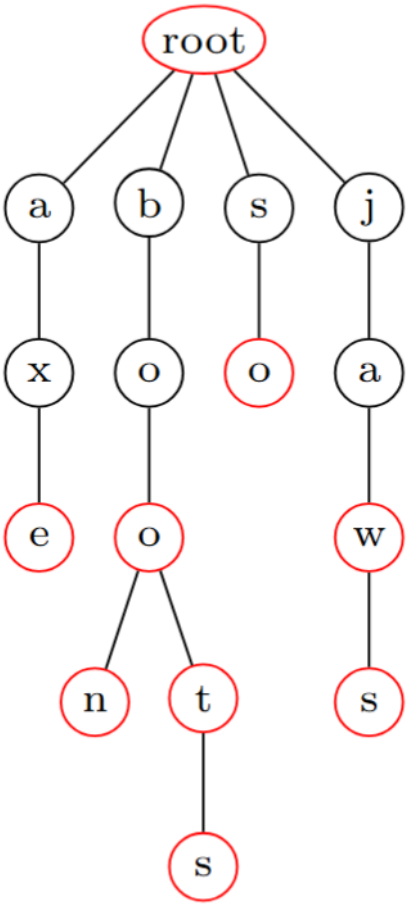
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ **Insert Jaw**



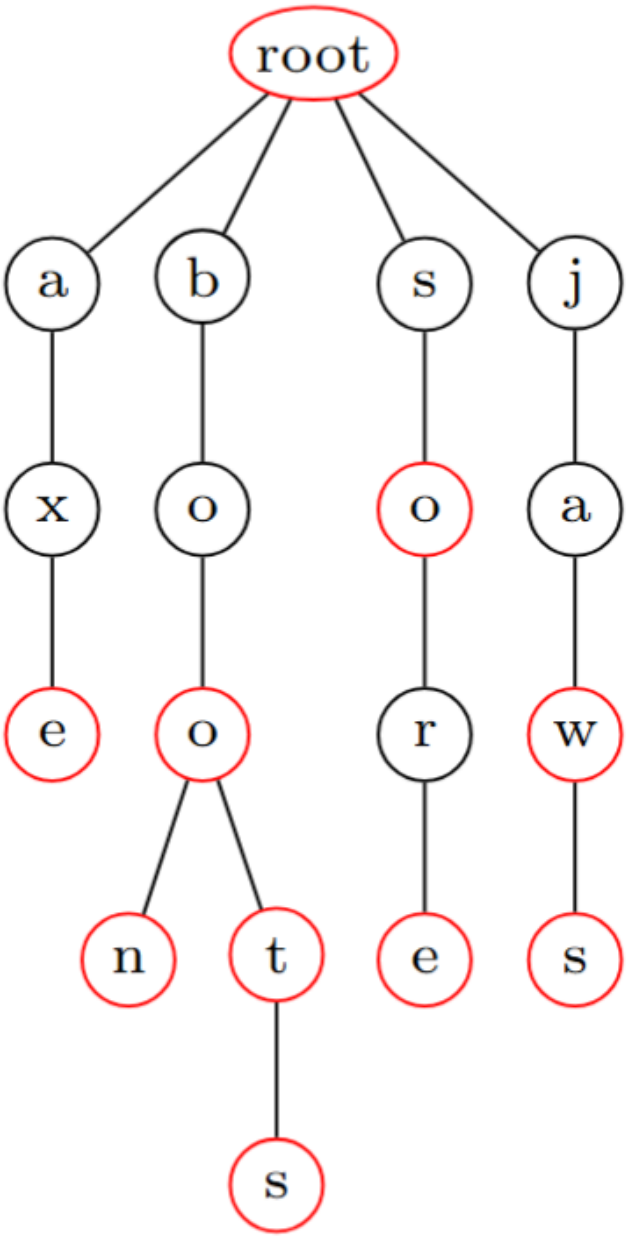
`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ Insert Boots



`so` `boo` `jaws` `boon` `boot` `axe` `jaw` `boots` `sore`

▼ Insert Sore



▼ Does the order that the words are inserted affect the shape of the tree?

The order of insertion does not affect the final shape of the tree.

▼ What are the two cases when deleting a word from a trie? How is each case handled?

The word ends at an internal node

The finishing (red) node is simply changed into a non-finishing (black) node

The word ends at a leaf node

The finishing (red) node is unlinked from its parent and removed (freed). If the parent of that node is a non-finishing leaf node, it is also removed from the tree. Repeat until we reach a finishing node or a non-leaf node.

▼ Autocomplete questions.

▼ What is autocomplete?

Autocomplete is a feature in which an application predicts the rest of a word a user is typing. For example, if a user has typed in "th", then autocomplete may suggest the words "the", "they", "then", and so on.

▼ Explain how a trie could be used to implement basic autocomplete, which suggests all possible words.

Fill a trie with words of the language then use search the sub tries for possible suggestions.

▼ Users are much more likely to type some words than others, so ideally autocomplete should suggest more likely words. For example, if a user has typed "tho", autocomplete should suggest "though" instead of "thou". How could you extend your solution so that autocomplete suggests more likely words?

Could store the count of each word and only choose the top 3, another way is to store the top three suggestions in the trie itself and update after usage, e.g. in node t store the top 3 suggestions and then so on do the storing only updating if clearly a lower count in a subtrie lapses the others into the top 3.

▼ Alien Poetry

It's the year 2521, and space exploration has uncovered an alien language (which just so happens to use the same letters as English 😉)! A team of linguists has found that these aliens loved poetry, and that two words are said to rhyme with degree k if the last k letters of each word are identical. For example, the words `GOODRA` and `QUOLODRA` rhyme with degree 4, as the last 4 letters of these words are identical. These words also rhyme with degree 1, 2 and 3.

To help investigate this poetry and write some of your own, you would like to devise a method to find the number of words that rhyme with a given word with degree k.

For example, suppose the words of the language are `ARON`, `AGGRON`, `SQUARON`, `RONA`, `LAVARON` and `UDON`. Then:

- The number of words that rhyme with `SQUARON` with degree 2 is 4 (the words are `ARON`, `AGGRON`, `LAVARON` and `UDON`)
- The number of words that rhyme with `SQUARON` with degree 3 is 3 (the words are `ARON`, `AGGRON` and `LAVARON`)
- The number of words that rhyme with `SQUARON` with degree 4 is 2 (the words are `ARON` and `LAVARON`)
- The number of words that rhyme with `SQUARON` with degree 5 is 0.

Can you come up with a method to efficiently answer queries like the ones above? Suppose you are allowed to preprocess the words of the language before answering any queries.

▼ Hint

Think about different ways of storing in a trie.

▼ Answer

Store in reverse order and traverse the last k letters then count the number of finishing nodes in the subtrie

▼ Is this efficient?

No, because you still need to go through an entire subtrie

▼ What should you do instead

- Store the number of ending nodes as you go, this can easily be done by adding to each node as you insert a new word. e.g. if there are 3 ending nodes in a subtree, store that at the node, this

can easily be maintained by updating as you go back up each time you add a new word, then you just need to check that number which is O(k)

- Now only need to go down to that node return the count - 1 because the current word is already stored in the trie too.
- O(k) time complexity where k is the degree.